

What you actually have.

Your goal, the formalization,
the kernels, and the possibilities.



Reflective Binary Field Computing

A readable consolidation of the visible revision history, with a literal operator-as-SDF construction, tested binary building blocks, and a map from present-day uses to futuristic applications.

Tom Klootwijk

NL200678942 | 10-07-1990

18 September 2026 · Consolidation C1
ELI5 main text · Mathematical appendix · Feasibility companion

Based on recoverable project messages and attachment labels. Original source PDF bodies and earlier kernel ZIPs were not accessible; new constructions and tests are labeled throughout.

Contents

1. What you actually have now	3
2. The literal source and revision ledger	4
3. What your goal was — in your own terms	6
4. How the revisions fit together	7
5. The formalization, explained without hiding the mechanism	9
6. One-bit words, the symbol, and retained information	10
7. What the GPU, texture atlas, and cache actually contribute	12
8. The kernels as supporting feasibility — what was actually checked	13
9. The numbers in ELI5 language	15
10. Present-day applications: where this could become useful	17
11. The nature and substance direction, made literal	19
12. Where it meets present-day frontier technology	21
13. Futuristic and science-fiction possibilities, with the missing bridge visible	22
14. What has been established, what remains a hypothesis	25
15. The development path and practical handover	26
Appendix A. The precise mathematical construction	28
Appendix B. Plain-language glossary	32
Appendix C. References and evidence register	34

Reading routes. For the main answer, read sections 1, 3–5, and 9–14. For literal provenance, read section 2. For what actually ran, read section 8. The equations are collected in Appendix A so the main explanation remains readable.

Reference key. H = visible history; D = attachment label; E = new code or obtained test evidence; R = external primary reference; A = construction in the mathematical appendix.

1. What you actually have now

You have a coherent direction for a **field-based computing language and runtime**: a system in which shapes describe not just objects, but also valid relationships, operations, and changes over time. Its intended engine is a GPU doing computation, not a renderer producing pictures.

Your repeated requests move toward one central idea: *put the description of the problem and the description of the rules into the same family of representations, then compile their useful decisions into densely packed binary work*. This interpretation comes from your visible project messages, rather than from an assumption that an SDF project must be about graphics. [H2–H6]

ELI5: Imagine a huge board of tiny yes/no lights. Some lights describe where things are. Others describe which combinations are allowed. A rule moves the board from one state to the next. Your ambition is to describe the board, the rules, and the clock using the same language, then let the GPU update many lights together.

There are three different things in your hands

The originating direction. The recoverable history establishes your requirements: signed-distance definitions, one-bit packing, operators represented as fields, logarithmic/polar lookup layouts, an explicit time operator, self-reference, GPU locality, and broader uses than calculation or rendering. It also records the traveling-salesman and nature/substance branches. [H1–H6]

A consolidated technical interpretation. This report makes those requirements compatible by separating the mathematical meaning of a field from the compact representation used to execute particular queries. It supplies a concrete operator-as-SDF construction, a rule-composition model, a time model, and an implementation path. These are explicitly *new consolidating constructions*, not quotations from the missing original formalization PDFs.

A new, limited feasibility companion. The accompanying source package implements and tests small parts of that interpretation. Its CPU tests pass. It includes CUDA source, but there was no CUDA compiler or GPU execution facility available here, so there are no measured results from your laptop. [E1–E3]

The resulting direction, aptly named

A useful descriptive name is **JSFD — Reflective Binary Field Computing**. Here “reflective” means that rules can be stored and addressed as data; “binary” describes the intended compact execution; and “field computing” distinguishes it from a graphics-only engine. This is a proposed name for the consolidated direction, not a recovered expansion of “JSFD” from your source files.

The most defensible summary is: **a research architecture with executable building blocks, not yet a demonstrated end-to-end platform for every proposed application.** That is a meaningful position. The next evidence needed is a complete compiler/runtime path and a useful application benchmark, not merely more terminology.

2. The literal source and revision ledger

You asked for the actual formalization, source documents, and kernels to be taken into account. The distinction between *a file being mentioned* and *its contents being available* matters here.

The accessible project history contains your requests and attachment names. Searching the uploaded-file index returned no files, and the working directory contained no earlier PDFs or ZIPs. The earlier assistant-generated formalizations and kernels were therefore not available for a line-by-line review. This report does not silently replace them with invented contents.

Scope of this edition: a consolidation of recoverable project intent, a literal ledger of the visible sources, and new supporting mathematics/code. It is not a verbatim restoration, a complete source-document synthesis, or a verified diff of the frozen releases.

Recoverable lineage — 18 September 2026

Record	Visible branch and what it establishes	What is not recoverable
H1 06:00	Formalize Definition PDF. Request to formalize your definition, preserve your identifiers, and incorporate the geometry/naming source and its enhanced version.	The initial formal definitions, diagrams, equations, and final PDF.
H2 07:46	Branch · Formalize Definition PDF. Request for a kernel for your RTX 5070 Ti 12 GB laptop, ELI5 metrics, and later one-bit packing, log-polar LUTs, cache/VRAM use, and .	The delivered kernel, its build state, benchmarks, and any tests.
H3 08:31	V5 FROZEN. Explicit clarification: operators and expressions should themselves be SDF definitions; the GPU is the calculating device, not a renderer. Time and self-reference become central.	The exact frozen V5 source tree and mathematical specification.

Record	Visible branch and what it establishes	What is not recoverable
H4 09:32	V5 1bit FROZEN. Reassertion of raw binary logic, one-bit field maps, bit shifts, and the same broader architecture.	Whether this branch changed storage alone, operator semantics, or the full execution path.
H5 10:13	V6 FROZEN. You reject a mere calculator/renderer interpretation and insist on the use cases in the ladybug/traveling-salesman source.	The full use-case list inside that PDF and the delivered V6 kernel.
H6 10:52	Biochemical geometries. Request to extend the formalization with the “bromofly...” source and literal mechanisms found in nature and substances.	The detailed chemical assertions, biological mechanisms, and completed expanded formalization.

Times are the branch labels visible in the project context, using the project’s Europe/Amsterdam timezone. They identify conversation records; they are not release-build timestamps. “Frozen” is preserved as your branch label, not treated as evidence of tested completeness.

Source-document names actually visible

D1. “men T room a circle a sphere a cone an apex legenda a side view of the pyramid...”

This initial attachment name is truncated in the available history. Its body is absent. It establishes a geometry/naming origin, not the meaning of each symbol.

D2. “men-T-room-a-circle-a-sphere-a-cone-an-apex-legenda-a-side-view-of-the-pyramid-lower-case-phi-log-encoded-polar-LUT-binary-search-tree-klein-bottl.pdf”

This is the visible enhanced-source filename. Terms in its name include lower-case phi, log-encoded polar lookup tables, binary search trees, and a Klein-bottle reference. Their intended connections cannot be recovered from the name alone.

D3. “ladybug-anti-violence-tile-in-zaandam-at-the-train-station-got-me-to-thinking-about-the-travling-salesman-NP-hard-problem-where-humans-try-to-solv.pdf”

A space-separated variant is also visible, continuing into “infinity by defining points instead of signed distance fields and packing thos...”. Both are recorded as visible labels, not assumed to be byte-identical versions. The traveling-salesman and point-versus-field themes are explicit; an exact application inventory is not.

D4. “bromofly and derivatives in nature insects bug finder 0 day signed distance field notations.pdf”

The filename is preserved literally. It is not evidence that a named chemical occurs in nature, that the label uniquely identifies a substance, or that a zero-day vulnerability was discovered. This report does not assign a molecular identity to “bromofly” or infer an exploit from the title.

What is credited to whom

The project direction and visible requests are attributed to **Tom Klootwijk | NL200678942 | 10-07-1990**, as requested. The synthesis, illustrative formal construction, and companion code in this edition are newly prepared supporting material. External references [R1-R16] establish neighboring technology and natural mechanisms; they are not presented as documents that necessarily appeared in your earlier source set.

3. What your goal was — in your own terms

The repeated clarifications are more reliable evidence of your goal than a generic description of what SDF software usually does.

“packing all operators as SDF definitions themselves not a rendering engine but just using the GPU as a calculating device” [H3-H6]

“all expressions as SDFs as well as 1bit words” [H3-H6]

“where A is T which in of itself is a defined time operator” [H3-H6]

“saturated the rest of the cache with self referential definitions of the formalization” [H3-H6]

“I am not looking for a calculator or a rendering engine, but for the usecases defined in the document” [H5]

The goal reconstructed from those statements

You wanted a common language for **objects, relationships, operators, expressions, and time**; a binary representation that avoids floating-point work where it is unnecessary; and an execution arrangement that keeps reusable information near the GPU’s compute units. The goal was not simply to make a prettier image of a mathematical surface.

You also wanted the architecture to become useful beyond its own demonstrations. The later revisions shift attention from “can this representation execute?” to “can it express the actual problems and mechanisms I care about?” That is the transition from an encoding experiment toward a domain-spanning computational framework. [H5–H6]

What should survive consolidation

The field-first description, first-class operators, binary execution, explicit time, and application-driven tests are central requirements. A particular texture format, a particular cache arrangement, or a claim that every task should use only OR is an implementation hypothesis that must be tested against those requirements.

ELI5: The destination is a machine that works from the same kind of “map” for both things and rules. A packed texture is one possible filing cabinet for those maps. The filing cabinet is not the destination.

What cannot be silently filled in

The visible history does not define what every geometric symbol means, give a complete grammar for the original formalization, or enumerate every use case inside D3 and D4. For example, this report does not assume that ϕ means the golden ratio rather than an angle or another project-specific symbol. The mathematical appendix gives a workable *new* interpretation of the recoverable architectural requirements without pretending to restore those missing definitions.

4. How the revisions fit together

Read as a sequence of requirements, the branches form a progression rather than a set of unrelated ideas. [H1–H6]

Layer	What it contributes	ELI5
Geometric descriptions	Describe a set through a rule that answers spatial questions, rather than only through a list of sampled points.	Keep the recipe for the shape, not only photographs of it.
One-bit representation	Pack yes/no outcomes and bit planes into machine words.	Store many tiny switches in one box.
Operators as fields	Describe valid input/output combinations as regions in a configuration space.	A rule is a map of which combinations are allowed.

Layer	What it contributes	ELI5
Time and self-reference	Store state transitions and references to reusable definitions explicitly.	The recipe can refer to other recipes and say what happens next.
Problem-solving	Connect representations to paths, constraints, candidate states, and measurable outcomes.	Use the maps to answer a useful question.
Nature and substances	Add domain-specific geometry, material properties, interaction rules, and dynamics.	Give the little switches rules that approximate a real mechanism.

A useful architecture has three distinct layers

Meaning. What does a field represent? A distance, a region, an allowed relation, a concentration, a time transition, or a program definition? These meanings are related but not interchangeable.

Representation. How is that meaning stored? As an analytic formula, a signed lookup table, a one-bit mask, several bit planes, or a graph of references? Several representations can describe the same allowed Boolean outcomes while preserving different amounts of geometric information.

Execution. Which GPU operations answer the particular query? They might be Boolean gates, integer address calculations, neighborhood propagation, or a more detailed numerical solver. Selecting a cheaper representation is useful only when the answer remains the answer the application needs.

The central design move: retain rich field definitions as the source of meaning, but execute a compact compiled form whenever its equivalence is established for the required queries.

This resolves the apparent tension between “everything is an SDF definition” and “the kernel should not spend its time calculating floating-point distances.” The source definition can be geometric while a particular compiled query is Boolean. The bridge must specify which queries it preserves. The next section supplies a literal, testable example.

5. The formalization, explained without hiding the mechanism

5.1 A shape answers a question

A signed-distance function, or SDF, answers: “How far is this location from the boundary?” In the convention used here, the answer is negative inside, zero on the boundary, and positive outside. A one-bit mask instead answers only: “Inside or outside?” They are different amounts of information.

An analytic circle can describe a continuous boundary with a small formula. Sampling that formula onto a finite grid gives a finite approximation. Keeping only each sample’s sign gives an even smaller representation. These are three stages, not three names for the same stored data. [A1]

5.2 An operator can literally be a shape

Take the rule **OR**: its result is 1 when either input is 1. Write the three bits as a location (a,b,c) , where c is the proposed result. There are eight possible locations because each of the three bits is either 0 or 1.

Exactly four are allowed: $(0,0,0)$, $(0,1,1)$, $(1,0,1)$, and $(1,1,1)$. Put a small solid cube around each allowed location. Make each cube half a unit wide, so the cubes do not touch. Their union is an ordinary geometric shape with an ordinary Euclidean signed-distance function.

At an allowed location, the SDF is $-\frac{1}{4}$. At a disallowed Boolean location, it is $+\frac{3}{4}$. Multiplying by four gives exact integer samples -1 and $+3$. This works for all sixteen possible two-input Boolean functions. The proof is in A2, and all 128 operator/vertex combinations were checked against an independent geometric formula. [E1]

ELI5: The correct answers sit inside little rooms. Wrong answers sit outside. To execute a rule, ask which proposed answer is inside a room. That is a literal “operator as a shape,” not just a metaphor.

5.3 Why the GPU can then use bits

For these legal Boolean questions, the sign is enough. The compiler can read the eight signed samples, identify the valid output for each input pair, and reduce the result to a four-bit truth table. A packed kernel then applies that rule to 32 independent cases stored in one 32-bit word.

This compilation preserves the operator’s Boolean behavior exactly. It does *not* preserve the complete distance function at every continuous location between the little rooms. The companion explicitly distinguishes those two claims. The achievement is exact behavior on a specified finite domain, not compression of arbitrary continuous information into one bit. [A2; E1]

5.4 Expressions combine relations

A longer expression connects outputs of one relation to inputs of another. For example, “first apply rule R, then rule Q” means that some intermediate value must satisfy both rules. In mathematical language, this is relation composition. In implementation language, it becomes a graph with typed inputs, outputs, and references. [A3]

A deterministic operator supplies one result for each allowed input. A constraint may allow several results, or none. A language that stores both must distinguish “execute the one result” from “search for a satisfying result.” Merely describing the allowed region does not provide the search strategy.

5.5 Time is a rule about state changes

A time operator says which next state is allowed after the current state. In the companion, every rule reads the old state, and the results are committed into a new state together. A rule may refer to its own previous output. That makes feedback meaningful without requiring a value to be known before it exists.

For example, “next state is NOT current state” makes a bit alternate 0, 1, 0, 1. It is a well-defined time evolution. As a zero-delay equation, “ $x = \text{NOT } x$ ” has no Boolean solution. An explicit time convention makes the difference. [A4; E1]

5.6 What self-reference adds

Giving definitions identifiers allows a program to reuse them, inspect their encodings, or eventually replace them. Storing a copy of an interpreter’s description is a further possibility. But the description still needs a running interpreter or compiled machine code to act on it. The current companion demonstrates delayed self-reference in state; it does not implement a complete self-hosting or self-modifying formalization.

6. One-bit words, the | symbol, and retained information

One bit per question is not one bit per complete number

A one-bit field is appropriate when the required answer is genuinely binary: occupied/free, allowed/forbidden, reached/not reached, true/false. When the application needs distance, concentration, speed, or a cost, it needs more information. One option is to store several bit planes, each carrying one digit of a fixed-width number.

The companion’s eight-plane adder computes 32 separate eight-bit additions at once using Boolean gates. It returns the low eight result bits and a carry bit for each case. All 65,536 possible pairs of byte inputs were tested. This demonstrates that a Boolean implementation can perform exact finite arithmetic; it does not establish that bit-sliced arithmetic is always faster than the GPU’s native integer instructions. [A5; E1]

There are three meanings of “|” that must not be mixed

Meaning	What it does	Consequence
C/C++ bitwise OR	$a \mid b$ sets a bit when either input has that bit set.	Excellent for joining compatible occupancy masks. Not, by itself, a universal Boolean basis.
Geometric union	Forms the set of locations inside either shape.	Its sign mask is the OR of compatible sign masks. Complete distance values require more care.
Sheffer stroke / NAND	Sometimes the same-looking symbol denotes NOT-AND in logic.	NAND alone can express all Boolean functions, but it is not the C/C++ OR instruction.

With ordinary positive-bit encoding, networks containing only OR, constants, copies, and routing cannot compute NOT: changing an input from 0 to 1 cannot make such a network's output drop from 1 to 0. NAND avoids that limitation. Its elementary completeness construction is given in A5.

ELI5: “Put the piles together” is a useful instruction. It cannot also mean “take the opposite” unless another mechanism is present. Using the same symbol does not make those operations identical.

A sign-correct union is not always an exact distance field

For two overlapping shapes, taking the smaller of their SDF values gives the correct union membership, but may give the wrong distance inside their overlap. A simple interval example in A1 shows why: an old internal edge is no longer a boundary of the combined shape. Packing signs is perfectly valid for membership queries; treating those signs as preserved physical distances is not.

This distinction keeps the binary direction intact. The right design is not “put floats back everywhere,” but **state which questions need only bits, which need multi-bit quantities, and which need a reconstructed metric.**

7. What the GPU, texture atlas, and cache actually contribute

The GPU is a compute device here

CUDA allows kernels to process data without rendering a scene. The companion's kernels produce logical states, sums, reachable sets, and lookup results; none requires a camera, rasterizer, image, or display loop. [R1; E2]

Integer textures are a real implementation option

NVIDIA documents `tex1Dfetch` with an integer index, and element-type reads can return integer texels without converting them to normalized floating-point values. The companion uses that path for signed operator samples and an atlas. Ordinary normalized/filtering texture operations are not interchangeable with this integer lookup path. [R2]

The implementation first stores all sixteen operators' signed vertex samples in a texture. A kernel reads the samples associated with output 1, derives the truth table, and applies it to a packed word. A second kernel uses the compact truth table directly. Comparing them tests whether the field representation's flexibility is worth its lookup overhead for that workload. [E2]

What the logarithmic/polar idea can mean

A logarithmic radial layout allocates bins by scale: near/far intervals grow multiplicatively instead of all having the same width. Angular bins describe direction. Such a layout may be useful when an application values relative distance and direction, but it sacrifices different detail than a uniform Cartesian grid. It is not a free compression theorem. [A6]

The companion provides a deliberately modest **integer log-radius/eight-sector atlas**. It computes a radius bin from the integer squared radius, chooses one of eight angular sectors, and combines those with one of four time phases. The origin has a separate convention. There are 1,024 atlas slots; many positions deliberately share a slot. [E2]

This is a newly defined feasibility example, not the recovered full polar LUT from D2. It still performs integer multiplication, comparisons, and address calculation. A finer precomputed angular LUT could trade more storage for less runtime calculation, but its error and memory cost would need to be specified.

"A is T" needs a data contract

Your request makes time part of the representation. A practical interpretation is to use a channel or metadata field to hold a time-state reference, phase, or transition identifier. That is different from merely treating an image's alpha component as a free extra coordinate. This report preserves the requirement that T be defined as an operator, while choosing an explicit four-phase address example only for the new demonstrator. [H3-H6; A4; A6]

Cache locality is useful; saturation is not the objective

NVIDIA exposes mechanisms that influence persistence in L2, subject to device limits. It does not promise unlimited residency, and competing working sets can evict one another. Filling the cache with duplicate definitions can crowd out the data that the current kernel actually needs. [R3]

A better implementation policy is to keep frequently reused rule tables and active tiles close to the compute units, while using VRAM for the larger state, alternative candidates, histories, and less frequently used definitions. The optimization target is **useful reuse and less data movement per answer**, not a percentage-full display.

Your stated RTX 5070 Ti Laptop GPU with 12 GB is consistent with NVIDIA's current product table. That does not identify your laptop's power configuration, free VRAM, or sustained kernel performance. The provided runner queries the actual device and memory properties instead of assuming those results. [R4; E2]

8. The kernels as supporting feasibility — what was actually checked

The companion is titled **JSFD consolidation feasibility companion**. It is new source prepared for this report. The historical V5/V6 kernels could not be recovered or inspected. A request for a kernel, or an earlier statement that one existed, is not substituted for a test log.

The seven CUDA kernels included

Kernel	What it tests	What it does not establish
or_kernel	A direct native packed-OR baseline.	A complete rule language or distance evaluator.
binary_gate_kernel	Execution of any of the sixteen two-input truth tables on packed words.	Efficient handling of every higher-arity relation.
relation_texture_kernel	Read signed operator samples through an integer texture and recover the same Boolean behavior.	Continuous SDF evaluation or a universal field interpreter.

Kernel	What it tests	What it does not establish
bitplane_add_kernel	Eight-bit addition using bit planes and Boolean carry propagation.	A speed advantage over native integer arithmetic.
reach_kernel	Obstacle-aware propagation through a closed rectangular grid.	A traveling-salesman optimizer or Euclidean shortest-path solver.
program_kernel	Synchronous state updates from a graph with delayed feedback.	Self-modifying programs or self-hosted compilation.
atlas_kernel	Integer log-radius/sector/time indexing and texture retrieval.	The exact historical atlas, lossless geometry, or cache pinning.

Actual results obtained in this environment

CPU verification	Coverage	Result
Literal operator SDF geometry	128 vertex checks: 16 operators × 8 vertices.	PASS
Packed Boolean execution	262,144 individual lane comparisons against truth-table results.	PASS
Eight-bit bit-sliced addition	All 65,536 input pairs, including carry-out.	PASS
Packing and partial-word tails	8,385 round-trip bit comparisons, lengths 0 through 129.	PASS
Grid reachability and arrival times	180 seeded random maps; 1,943,045 cell/time comparisons against an independent breadth-first search.	PASS
Time and delayed feedback	64 synchronous steps, including a period-two self-referential rule.	PASS
Coarse atlas addressing	66,564 coordinate/phase checks, plus a large-coordinate boundary check.	PASS

Source: actual output in `evidence/host_results.json` [E1]. Compiler: `g++ 14.2.0` on Linux x86-64 [E3]. Random seed: 20260918. These are counts of checked cases, not speed, accuracy-on-real-world-data, or scientific validation scores.

The evidence boundary: CPU compilation and semantic tests were performed. CUDA compilation and execution were not: no `nvcc` or `nvidia-smi` executable was available. The CUDA source is supplied for testing, not described as an already successful GPU run.

Why the CPU tests still matter

They show that the finite mathematical construction and packed algorithms produce the intended results on the tested domains. They catch errors such as bit-order mistakes, wraparound at grid edges, wrong carry propagation, and ambiguous time updates. The same core functions are shared with the CUDA source, making this a useful semantic baseline. Device compilation, memory behavior, scheduling, and performance still need their own tests.

The independent geometric oracle uses floating-point math only to check the source SDF formula. The kernel data path remains integer/Boolean. CUDA's host-side timing API also reports floating-point milliseconds; this is instrumentation, not floating-point field arithmetic. [E1-E3]

9. The numbers in ELI5 language

A bit mask can save a lot of space

For a field with N binary answers, packed storage needs approximately $N/8$ bytes, rounded up for the final word and any row padding. Storing each answer in a 32-bit float takes 4N bytes. That is a **32-fold reduction in payload storage for the same binary information**, not a 32-fold speed measurement. [A7]

A $512 \times 512 \times 512$ field	Payload size	Information retained
One packed bit per cell	16 MiB	One yes/no answer.
One byte / eight bit planes per cell	128 MiB	256 possible states or eight separate flags.
One 32-bit float per cell	512 MiB	A numerical value with floating-point range and precision.

A $512 \times 512 \times 512$ field	Payload size	Information retained
Four one-bit time slices	64 MiB	Four snapshots of one binary answer.

Calculated directly: $512^3 = 134,217,728$ cells; 1 MiB = 1,048,576 bytes. Sizes exclude metadata, padding beyond this particular grid, program tables, temporary buffers, and allocator overhead.

What 12 GB does and does not mean

An idealized 12 billion-byte budget holds 96 billion bits. An idealized 12 GiB budget holds about 103.1 billion bits. Neither is a usable whole-system cell count: the runtime, other applications, program definitions, inputs, outputs, and intermediate states also occupy memory. The runner reports bytes from the actual device rather than assuming one convention. [A7; E2]

If each modeled location requires eight bit planes and three state buffers, it consumes 24 bits before other overhead. The same memory then holds only one twenty-fourth as many modeled locations as a single-bit/single-buffer estimate. Keeping all variables visible prevents impressive but misleading capacity claims.

Thirty-two bits at once is useful, but not a promised speedup

A 32-bit OR instruction can combine 32 pairs of logical lanes. Real execution also has loads, stores, addressing, rule decoding, dependencies, and scheduling. A texture lookup can cost more than the arithmetic it replaces. If a workload is already handled efficiently by native packed words, adding a geometric interpretation may improve expressiveness without improving raw speed.

ELI5: Carrying 32 marbles in one box can reduce the number of trips. It does not make the elevator 32 times faster, and labeling every marble may take extra time.

The comparison that would be meaningful

Measure an end-to-end answer against an equally correct baseline: ordinary GPU bitsets for binary propagation, native integer arithmetic for finite sums, an established geometry library for spatial distances, and a suitable optimizer for route quality. Include encoding and decoding costs. Report both answer quality and total time, not only an isolated inner-loop instruction count.

The companion's timing runner reports warm-cache medians for two implementations of the same OR result. Its computed payload rate counts logical input/output bytes; it is not a direct measurement of physical DRAM traffic. No performance ratio is filled in before the GPU run exists. [E2]

10. Present-day applications: where this could become useful

This section separates what your messages explicitly establish from new application translations. The traveling-salesman direction is explicit in D3/H5. The other examples below are **proposed extensions of the architecture**, not a recovered list from your missing PDF.

10.1 Routes, reachability, and constrained movement

ELI5: Flood a maze with ink. The places the ink reaches are connected to the start. The moment a place first gets wet tells you its shortest step count on a uniform four-neighbor grid.

The companion already tests that finite version. A useful application could batch many occupancy maps, candidate starting states, or obstacle variations. For a physical robot, the field must also account for the robot's size, orientation, articulation, and permitted motion. A bit indicating that a point is free is not enough to guarantee that a whole moving robot fits.

cuRobo is a relevant existing comparison because it exposes robot/world collision-distance and motion-planning functionality. Its documented signed-distance convention differs from this report's convention, illustrating why interface contracts matter. [R5]

Next proof: a fixed set of maps, independently checked paths, obstacle clearance, update latency, and an ordinary GPU bitset baseline. This would demonstrate a useful navigation component, not yet a full robot controller.

10.2 Traveling-salesman and scheduling problems

ELI5: A maze asks how to get from one place to another. Traveling-salesman asks how to visit all the required places in a good order and return. Those are different jobs.

A field representation can encode allowed positions, feasible route fragments, collision constraints, or collections of candidate solutions. It may accelerate testing and propagation. But the choice of a visit order, the cost of that order, and the prevention of disconnected subtours still have to be represented and handled.

Replacing points by fields changes the representation; it does not by itself prove that an exact search becomes easier. A specific algorithm could improve practical performance, so the right experiment measures route quality and total cost of obtaining it. Concorde provides a relevant exact symmetric-TSP reference. [R16; A3]

Next proof: small instances with known optima, then larger matched instances; disclose whether the method is exact, heuristic, or approximate. The current wavefront kernel is supporting infrastructure, not this optimizer.

10.3 Design spaces, collision envelopes, and manufacturability

ELI5: Instead of checking one design at a time, mark whole regions of designs as allowed or forbidden.

Examples include fitting components into an enclosure, reserving clearance around moving parts, or screening coarse candidate geometries before detailed analysis. Packed masks can cheaply eliminate impossible candidates. A later metric stage can recover distances, tolerances, and surface quality for the survivors.

NanoVDB is relevant prior technology for compact GPU-accessible volumetric data and collision-related workflows. It demonstrates that field representations can be practical without establishing that this new architecture outperforms them. [R6]

Next proof: compare the same geometric queries at the same spatial resolution, including false-free and false-blocked classifications near thin features. Memory savings that erase a necessary clearance are not useful savings.

10.4 Bounded verification and software testing

ELI5: Treat a program's possible states as a maze, and ask whether any path reaches a forbidden room.

Finite relations and bitsets can represent allowed transitions and sets of states. A prospective JSFD frontend could turn a bounded controller or protocol into that representation and test reachability of a specified failure condition. "Bug finder 0 day" in D4 is only a title-level hint; it does not establish that this was the intended mechanism or that any vulnerability was found.

Next proof: encode a small system with independently known reachable states and deliberately inserted errors, then compare results with direct enumeration or an appropriate established verifier. State-space size can grow rapidly; the representation must expose rather than conceal that cost.

10.5 Local simulation and pattern systems

ELI5: Give each square a small rule about its neighbors, then watch a larger pattern emerge.

The temporal graph and neighborhood propagation kernels are relevant building blocks. They could support cellular automata, local constraint relaxation, and toy growth models. A physically meaningful simulation additionally needs quantities, units, conservation laws where appropriate, and validation against the phenomenon being modeled.

Next proof: one precisely defined local model with checked updates, known boundary conditions, and a clear distinction between a binary toy model and a quantitative scientific model. A simulated pattern can be useful even before it is a faithful account of a natural mechanism.

11. The nature and substance direction, made literal

The nature branch should not reduce to “it looks like an insect, so it must compute like one.” A literal translation specifies **material, geometry, interaction, state, and observable behavior**. The examples here are newly researched connections to H6, not claims that their detailed mechanisms appeared in D4.

11.1 Pine-cone scales: material geometry that moves with moisture

Research on pine cones describes moisture transport and the different swelling behavior of structural regions that produce opening and closing. Cellulosic structure and orientation matter to the resulting motion. [R11]

Field translation: a shape field describes each scale; a moisture field describes its internal condition; a material-orientation field helps determine its deformation. The state update maps moisture changes to a predicted new shape. A yes/no threshold could model “open enough,” while accurate curvature requires richer quantities.

ELI5: Two attached layers change size differently when they get wet, so the combined piece bends. The opportunity is a compact model for responsive geometry, not a claim that an SDF alone makes an actuator.

11.2 Gecko attachment: tiny contact geometry and interaction forces

Experiments on gecko setae provide evidence for van der Waals contributions to dry adhesion, with small contact structures playing a central role. [R12]

Field translation: a shape field describes the contact tips and surface, while separate material/contact parameters determine the interaction. The system could screen contact configurations or approximate an adhesion envelope. An SDF supplies geometric separation; it does not supply the correct adhesive force law by itself.

ELI5: Many tiny contact points behave differently from one large contact point. Geometry helps decide where contact happens, and the material model decides what that contact does.

11.3 Slime-mold networks: local adaptation and global transport

Tero and colleagues reported adaptive networks formed by slime mold with useful trade-offs between transport efficiency, cost, and fault tolerance, and described a mathematical model motivated by that behavior. [R10]

Field translation: candidate links can be represented by masks, while link capacity, flow, or reinforcement uses additional state. A proposed update can favor useful connections and let unused ones weaken. The outcome should be evaluated on network cost and reliability rather than presented as an automatic exact traveling-salesman solution.

ELI5: A transport network can change its own roads using local feedback. That suggests a family of heuristics, not a shortcut that guarantees the best route for every problem.

11.4 DNA: molecular interactions implementing logical behavior

Qian and Winfree demonstrated digital circuits using DNA strand-displacement mechanisms, including multilayer logic and a four-bit square-root circuit. Their work included a hierarchy between circuit descriptions and molecular implementations. [R13]

Field translation: finite logical relations can describe the intended input/output behavior. A separate physical model must account for binding, competing interactions, concentrations, and timescales. Your common language could become an intermediate description shared by a simulator and a future molecular-design backend.

ELI5: The same logical rule can be played by electronic switches or by carefully engineered molecular interactions. Writing the rule is not the same job as building the molecules that reliably play it.

11.5 Growth, repair, and reaction-like local updates

Growing Neural Cellular Automata demonstrated learned local update rules that can form target patterns and, under suitable training, recover from forms of damage. This is evidence about computational pattern dynamics, not proof of general biological regeneration. [R9]

Field translation: a JSFD model could store local states in bit planes and describe candidate rules as relations. A meaningful experiment would test growth, stability, and recovery after damage. Binary execution of a learned rule is a further compilation problem; it is not already supplied by the companion's NOT/XOR time example.

Where molecular shape stops being enough

Substance behavior can depend on forces, charges, bonding, conformations, environment, and dynamics. OpenMM's documented force models illustrate the additional numerical structure involved in molecular simulation. An SDF can contribute geometric questions such as separation or overlap, but cannot replace all those model terms merely by being called a field. [R15]

The useful direction is therefore **geometry plus explicit interaction and state laws**. That formulation allows serious nature-inspired work without silently equating a suggestive shape, an actual material, and a validated biological mechanism.

12. Where it meets present-day frontier technology

These are independently documented neighboring directions, checked for this report on 18 September 2026. They are evidence that parts of the design space are active and technically meaningful, not demonstrations of your complete architecture.

Compact neural and volumetric fields

Instant Neural Graphics Primitives uses multiresolution hash encoding with small neural networks, including an SDF task. It is a concrete example of co-designing representation and GPU execution. Its learned feature values and numerical computation differ from your strict binary direction. [R7]

Possible JSFD connection: use a compact learned or analytic field to generate certified binary decisions for a restricted query set, or use binary masks as a coarse stage before detailed field evaluation. The test would compare total query cost and error, not assume that replacing numbers with signs preserves the full task.

Learned logic-gate networks

Convolutional Differentiable Logic Gate Networks, published at NeurIPS 2024, extends learning methods whose eventual inference is expressed using logic gates such as NAND, OR, and XOR. This is a close conceptual neighbor to efficient binary execution. [R8]

Possible JSFD connection: learned rules could populate the operator library, with a compiler mapping them to packed execution. A research question is whether field-based constraints or geometry improve the training target or explain the result. The current kernel includes no learning algorithm, and the use of Boolean inference does not imply a floating-point-free training procedure.

Programmable local dynamics

The neural-cellular-automata work offers a concrete route from local state rules to larger patterns. [R9] A JSFD implementation could explore whether a restricted, quantized rule language preserves the desired behavior while lowering storage or inference cost. This would require a training or rule-search pipeline and an appropriate quality benchmark.

A genuinely current molecular-computing connection

Stérin and colleagues' *A thermodynamically favoured molecular computer* was published online in Nature on **16 September 2026**, in the issue dated **17 September 2026**. The paper reports a scaffolded DNA computing system tested on ten programs, including parity and arithmetic, in which the intended output is thermodynamically favored. [R14]

Why it matters here: it is a literal example of designing a physical system so that allowed configurations and dynamics implement computation. The connection to your direction is conceptual: a field/rule description might become a simulation or compilation layer for such a system. Your GPU software does not thereby inherit its physical behavior, energy characteristics, or experimental results.

Source review for R14 used the article's parsed text and bibliographic information. The publisher's PDF screenshot endpoint failed; no figure-based measurements are used in this report.

The strongest research intersection

Common descriptions, multiple backends: one typed relation/field language could target a packed GPU interpreter today, a specialized logic circuit later, and eventually a separately validated physical implementation. The difficult and potentially valuable contribution is the correctness-preserving translation between those levels.

That is a more specific opportunity than “an SDF replaces all computing.” It gives the project a measurable target: demonstrate that a useful class of problems can move through the shared language with less storage, less repeated work, or better interoperability than the alternatives.

13. Futuristic and science-fiction possibilities, with the missing bridge visible

The distinction here is not “impossible because futuristic.” It is the difference between a possibility supported by current ingredients, a research program requiring new integrations, and a story whose essential physical mechanism has not been specified.

Possibility	How this direction could contribute	What still has to exist
A programmable digital world	One representation for geometry, allowed interactions, local rules, and time. Visualization can be a separate consumer.	A complete language, scalable runtime, content tools, meaningful physics or game rules, and debugging.

Possibility	How this direction could contribute	What still has to exist
A self-repairing virtual material	Local rule fields that recover a target structure after virtual damage.	A successful learned or designed repair rule, stability tests, and clear limits on the damage it can repair.
A shape-changing robotic material	A compiler/simulator describing desired configurations and local coordination.	Physical modules, power, sensing, communication, actuation, tolerances, and distributed control.
An inverse-design laboratory	Search through geometry and rule combinations for a desired behavior, rejecting impossible candidates quickly.	Validated physical models, a search method, objective functions, and experimental checks.
One program on GPU, FPGA, or custom silicon	Finite relations can be lowered to Boolean circuitry as well as software kernels.	Backend-specific compilers, resource models, timing closure, and equivalence tests.
A molecular-computing frontend	Describe the logical behavior in a common intermediate form before a physical backend is chosen.	A chemistry-specific compiler and measured reliability, selectivity, throughput, and readout.
A rich “holodeck” simulation	The computation layer could provide a coherent world state to visual and interactive systems.	Display, optics, tracking, haptics, and physical force generation. The field definition alone supplies none of these.

Possibility	How this direction could contribute	What still has to exist
A large autonomous planning system	Spatial memory, constraints, candidate-state propagation, and rule representations could be components.	Learning, models, objectives, uncertainty handling, interfaces, and evidence of actual intelligent behavior.

The near-to-far progression

Closest to the current code: bitset computations, finite rule networks, toy local dynamics, and coarse reachability. These already have well-defined small demonstrations in the companion.

Next integration step: an application-specific field language and compiler that make those kernels useful without manually wiring every test. This is where the project could begin to function as a real tool rather than a collection of kernels.

Further research: learned rule discovery, richer physical models, inverse design, and equivalent backends. These add new scientific and engineering content, not just more VRAM.

Physical science-fiction scale: programmable matter, autonomous repair of real structures, and large distributed synthetic systems. The architecture could be a modeling or programming contribution, but the physical devices and validated mechanisms remain separate achievements.

What does not follow merely from the notation

Self-reference does not establish consciousness. Compact formulas do not provide infinite memory. A symbolic time operator does not manipulate physical time. A finite-state simulator does not create matter or energy. A geometric problem encoding does not, by itself, supply an efficient optimizer. These statements identify missing mechanisms; they do not prevent research into a specific, testable mechanism.

ELI5: A language for describing buildings can help design a remarkable city. It is still necessary to build the roads, power stations, and buildings. Your direction could become a common design language for computational systems; each physical application must supply its own machinery.

14. What has been established, what remains a hypothesis

Claim	Status in this edition	Needed for the next level
The intended project is broader than rendering.	Established by your visible requests. [H3–H6]	No further inference is needed about that intent.
Finite Boolean operators can be given literal SDF definitions.	Constructed and checked here for all two-input Boolean functions. [A2; E1]	Expand to the required typed operators without impractical table growth.
The defined Boolean queries can run on packed words.	CPU-tested here. CUDA source supplied. [E1–E2]	GPU build, execution, and profiling.
The original V5/V6 kernels prove feasibility.	Not assessable from the accessible record. Their code and logs are absent.	Exact source artifacts, build information, and reproducible tests.
The full formalization compiles into a general runtime.	Architecture proposal. No full parser, compiler, or self-hosted interpreter is included.	An end-to-end example generated from the actual language.
Log-polar storage is better than other layouts.	Workload-dependent hypothesis. The coarse atlas has defined behavior, not a demonstrated advantage.	Equal-quality comparisons against Cartesian, tiled, sparse, and alternative layouts.
Cache/VRAM saturation improves performance.	Not demonstrated. Useful reuse, not occupancy alone, is the relevant target.	Measured cache behavior, traffic, occupancy, and end-to-end latency.
The method solves traveling-salesman instances effectively.	Application direction. No TSP optimizer is in the companion.	A defined solver, route-quality results, and suitable baselines.
The nature branch predicts real substance behavior.	Modeling direction. Natural mechanisms are referenced, not simulated or experimentally reproduced here.	Calibrated models, correct units, and independent physical validation.

What may be distinctive

The candidate contribution is the *specific integration*: fields as a shared language for geometry and relations, compact query-preserving compilation, temporal/reflexive definitions, and GPU-aware layouts. The individual ingredients have neighbors in existing field methods, logic networks, local automata, and GPU data structures. [R1; R6–R9]

This report does not establish a historical priority claim or verified uniqueness. To establish a technical distinction, specify what this combination does that the closest alternatives do not, then demonstrate it under controlled comparison.

A useful success criterion

The most informative first result would be a real problem expressed once in the proposed language, compiled automatically, run correctly on the GPU, and compared with a strong baseline including translation costs. Even one such case would support the architecture more directly than a large catalog of speculative applications.

15. The development path and practical handover

Step 1 — preserve the source lineage

Maintain a separate source archive for D1–D4, each formalization revision, and each original kernel ZIP. Record filenames, hashes, dates, and which document changes which definition. Keep the frozen V5/V6 labels intact. This edition should be labeled **Consolidation C1**, not silently called a verified V7 or replacement for the absent originals.

Step 2 — write the smallest complete language contract

Specify field types, sign conventions, coordinate domains, physical units where relevant, bit ordering, operator interfaces, expression composition, time updates, and the meaning of cyclic references. Include an example that begins as a source expression and ends as a checked output. The appendix is a starting construction, not a substitute for recovering project-specific symbols from D1/D2.

Step 3 — run the companion on the laptop

The ZIP contains CPU tests, CUDA source, CMake configuration, and a README. The CPU default requires a C++17 compiler. The optional GPU build requires a compatible CUDA toolkit and host compiler; it detects the available GPU through the native architecture setting. [E2]

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build --config Release
ctest --test-dir build -C Release --output-on-failure

cmake -S . -B build-gpu -DJSFD_BUILD_CUDA=ON \
  -DCMAKE_BUILD_TYPE=Release
cmake --build build-gpu --config Release
./build-gpu/jsfd_cuda 262144 > gpu_results.txt
```

These are Linux/WSL-style run paths. The README includes Windows multi-configuration executable paths. The numeric argument is a count of 32-bit words, not bytes. The runner does not fill all VRAM.

Step 4 — turn one application into the first complete vertical test

For the recoverable direction, a constraint/reachability task is a sensible proposed first target: its correct outputs can be checked exactly, and its binary representation is natural. Then connect it to route-candidate screening or a small domain-specific model. Keep that choice distinct from claiming it was the complete original use-case list.

A benchmark report should include the source problem, compiled representation size, preprocessing time, kernel time, output decoding time, peak memory, correctness or approximation error, and hardware/software versions. Measure direct packed baselines as well as more elaborate field implementations.

Step 5 — test the architecture's signature features

Compare field-texture interpretation with compiled gates, log-polar layout with other layouts, and compact references with duplicated definitions. Add temporal histories and rule changes only after their semantics are explicit. This isolates which proposed features help rather than treating all revisions as a single untestable bundle.

What is delivered with this report

The PDF: your goal, the visible source/revision ledger, an ELI5 explanation, a new literal mathematical construction, current application connections, the nature branch, future possibilities, and an evidence inventory.

The ZIP: source for the new finite demonstrators and seven CUDA kernels, a build/run guide, actual CPU results, environment information, and a hash manifest. It does not contain the unavailable original source PDFs or earlier kernels.

Appendix A. The precise mathematical construction

This appendix makes the plain-language account checkable. These equations are new consolidating definitions for this edition, not a transcription of missing prior PDFs.

A1. Geometric fields and their signs

Let S be a nonempty proper regular closed subset of Euclidean space with nonempty boundary. Define:

$$\begin{aligned} d_S(x) &= -\text{dist}(x, \partial S) \text{ for } x \in S; \\ d_S(x) &= +\text{dist}(x, \partial S) \text{ for } x \notin S. \end{aligned}$$

The sign mask on chosen sample locations is:

$$M_S(x) = 1 \text{ when } d_S(x) \leq 0, \text{ and } 0 \text{ otherwise.}$$

Boundary inclusion is explicit. The operator construction below has no sample centers on a boundary, so using < 0 there gives the same Boolean classification.

On the same sample domain, union membership satisfies $M(S \cup Q) = M(S) \text{ OR } M(Q)$. This statement preserves membership, not all distances. For example, take $S = [-2, 1]$ and $Q = [-1, 2]$. At $x = 0$, both distances are -1 , while the distance to their union $[-2, 2]$ is -2 . Thus the minimum of the two input SDF values is not the exact union distance at that location.

A2. A literal SDF for every two-input Boolean operator

Let $f: \{0,1\}^2 \rightarrow \{0,1\}$. Its valid relation is:

$$R_f = \{ (a,b,c) \in \{0,1\}^3 : c = f(a,b) \}.$$

Around each vertex $v \in R_f$, place the axis-aligned closed cube:

$$\begin{aligned} C_v &= v + [-1/4, 1/4]^3, \\ S_f &= \bigcup_{v \in R_f} C_v. \end{aligned}$$

These cubes are separated: distinct Boolean vertices differ by one unit in at least one coordinate, while each cube extends only one quarter-unit in either direction. For $q = |x - v| - (1/4, 1/4, 1/4)$, the cube SDF is:

$$d_{Cv}(x) = \|\max(q, 0)\|_2 + \min(\max(q_1, q_2, q_3), 0).$$

Because the cubes are disjoint and separated, their union has exact SDF:

$$F_f(x) = \min_{v \in Rf} d_{Cv}(x).$$

At a valid vertex, x lies at the center of its cube, so $F_f(x) = -1/4$. At an invalid Boolean vertex, changing only c gives a valid vertex one unit away. The closest point on that cube is $3/4$ unit away. No valid cube is closer, because every different Boolean center differs by at least one unit in some coordinate. Therefore:

$$\begin{aligned} 4F_f(a, b, c) &= -1 \text{ if } c = f(a, b); \\ 4F_f(a, b, c) &= +3 \text{ otherwise.} \end{aligned}$$

This gives exact integer samples of a literal Euclidean SDF. Number the vertices by $v = 4a + 2b + c$. The output truth table is recovered from the signs at vertices with $c = 1$:

$$L_f = \sum_{a, b \in \{0, 1\}} 2^{2a+b} \cdot 1[F_f(a, b, 1) < 0].$$

The resulting four-bit number determines every legal Boolean query. The companion independently checks the box formula and the recovered truth table for all sixteen f . The construction proves representability; it does not claim geometric evaluation is the fastest way to execute a gate.

A3. Expressions, relations, and the cost of projection

For finite domains X, Y, Z , let $R \subseteq X \times Y$ and $Q \subseteq Y \times Z$. Composition is:

$$(Q \circ R)(x, z) \Leftrightarrow \exists y \in Y : R(x, y) \wedge Q(y, z).$$

In words: there is a connecting intermediate value that both rules allow. If R and Q are total deterministic functions, execution follows their unique outputs. If they are general constraints, finding or enumerating suitable y may require search. A sign field for the allowed tuples is a representation of this relation, not a proof of efficient projection.

A raw truth table for k input bits has 2^k entries. For a function of two n -bit numbers, there are 2^{2n} input combinations before storing the outputs. Structured circuits, shared subexpressions, sparse relations, and domain-specific decomposition are therefore essential. Storing every allowed combination is not the proposed general implementation.

A4. State, time, and reflective definitions

A proposed typed machine state is:

$$\begin{aligned}\Sigma_t &= (X_t, P_t, A_t, t), \\ \Sigma_{t+1} &= T(\Sigma_t).\end{aligned}$$

X holds data fields; P holds definitions/program references; A describes address maps and lookup payloads; t is the logical step. A transition can itself be expressed as a relation $R_T(\Sigma, \Sigma')$. The companion restricts this model: P is fixed, all nodes read X_t , and all outputs form X_{t+1} . It does not evolve the full definition store.

For node i with input references $a(i)$, $b(i)$ and truth table $L(i)$:

$$X_{t+1}[i] = \text{Eval}(L(i), X_t[a(i)], X_t[b(i)]).$$

Self-reference $a(i)=i$ is valid because it reads the previous time step. Every finite deterministic closed state system eventually repeats a state, but it need not settle to a fixed point; cycles are allowed. A repeating phase index is not an unbounded physical clock.

For actual reflective execution, additional operations would need to encode, decode, inspect, and update P while preserving valid types and references. They are a specified future language feature, not something caused by placing bytes in cache.

A5. Packed execution and a complete Boolean basis

For four-bit truth table L , define m_j to be an all-zero or all-one word depending on bit j of L . For packed input words A and B :

$$\begin{aligned}\text{Eval}(L, A, B) &= (\neg A \wedge \neg B \wedge m_0) \vee (\neg A \wedge B \wedge m_1) \\ &\vee (A \wedge \neg B \wedge m_2) \vee (A \wedge B \wedge m_3).\end{aligned}$$

Each word bit is an independent lane. No cross-lane carry occurs in these logical operations. Multi-bit addition uses separate planes and explicit carry:

$$\begin{aligned}
 p_k &= a_k \text{ XOR } b_k; \\
 s_k &= p_k \text{ XOR } c_k; \\
 c_{k+1} &= (a_k \text{ AND } b_k) \text{ OR } (p_k \text{ AND } c_k), \text{ with } c_0 = 0.
 \end{aligned}$$

If the desired universal symbol is NAND, write $N(a,b)=\text{NOT}(a \text{ AND } b)$. Then $\text{NOT}(a)=N(a,a)$, $\text{AND}(a,b)=N(N(a,b),N(a,b))$, and $\text{OR}(a,b)=N(N(a,a),N(b,b))$. This proves a complete Boolean basis. It does not make C/C++ | mean NAND.

A6. Log-radius addressing and metric bookkeeping

For the new companion, x and y are signed integers with $|x|, |y| \leq 2^{30}$. Compute $r^2=x^2+y^2$ in a 64-bit unsigned range. For $r^2>0$:

$$\begin{aligned}
 \text{ring} &= 1 + \lfloor \log_2(r^2) \rfloor / 2; \\
 \text{address} &= ((\text{phase} \cdot 32 + \text{ring}) \cdot 8) + \text{sector}.
 \end{aligned}$$

The integer implementation obtains the inner logarithm by shifts. $\text{phase}=t \bmod 4$; sector is one of eight octants selected by signs and coordinate comparisons. At the origin, $\text{ring}=0$ and $\text{sector}=0$. The code defines ties on axes and diagonals deterministically. The representation is intentionally many-to-one.

For a continuous conceptual map with $\rho=\ln r$ and angle θ , Euclidean length is:

$$ds^2 = e^{2\rho}(d\rho^2 + d\theta^2).$$

Thus ordinary distance in the (ρ, θ) grid is not Euclidean distance in the original plane. Any use of polar/logarithmic sampling for physical distance or derivatives must account for that metric and its discretization. Addressing alone does not preserve it.

A7. Memory accounting

A dense one-bit sequence of N entries uses $4\lceil N/32 \rceil$ bytes in 32-bit words. A row-padded $W \times H$ grid uses $4H\lceil W/32 \rceil$ bytes per plane. With B bit planes and K state buffers, multiply that amount by BK before adding metadata and temporaries.

The exact 512^3 example in section 9 has no partial-word waste because 512 is divisible by 32. The nominal ratio to one 32-bit value per cell is $32/B$ for B bit planes, assuming equal numbers of buffers and ignoring other overhead.

Appendix B. Plain-language glossary

Term	Readable meaning
Field	A rule or stored map that assigns information to a location or configuration.
Signed-distance function / SDF	A distance to a boundary, with the sign distinguishing inside from outside.
Implicit description	A definition that answers whether a location belongs to a shape instead of listing every surface point.
Occupancy/sign mask	A yes/no version of a field. It does not store how far something is from a boundary.
Relation	A list or rule describing which combinations of values are allowed.
Configuration space	A space whose coordinates describe a complete candidate situation, not necessarily a physical position.
Operator as a field	An allowed region encoding a rule's correct input/output combinations.
Truth table	The output of a Boolean rule for every possible input combination.
Bit packing	Storing multiple independent yes/no values in one machine word.
Bit slicing / bit planes	Putting the same binary digit of many numbers together so gates can process them in parallel.
Lookup table / LUT	Stored answers retrieved by an index instead of recomputed each time.
Atlas	An organized collection of fields or table regions, together with a rule for addressing them.
Log-polar	A layout using direction and multiplicative distance scales rather than ordinary equal-width x/y spacing.
Kernel	A function launched to run many pieces of work on a GPU.
VRAM	The GPU's larger device-memory store.

Term	Readable meaning
Cache	A smaller, faster store for data the hardware expects to use again.
Compiler	A translator from a source description to instructions or another executable representation.
Interpreter	A running program that reads and executes encoded rules.
Reflection	A program's ability to represent or inspect descriptions of its own rules.
Delayed self-reference	A rule referring to its own value from a previous step.
Fixed point	A state that remains unchanged when the update rule is applied.
Heuristic	A method intended to find useful solutions without necessarily proving the best possible answer.
Backend	The particular execution target: a CPU, GPU, circuit, or separately engineered physical system.

One-sentence conclusion: Your direction is a common field language for things, rules, and change, with compact binary execution as its intended practical engine; the key next achievement is a useful end-to-end application whose correctness and cost can be measured.

Appendix C. References and evidence register

All external sources were consulted for this consolidation on 18 September 2026. These references support specific neighboring methods and mechanisms, not the performance of the JSFD companion. No third-party source PDF is reproduced or bundled.

Project and newly generated evidence

H1-H6: visible project conversation records described in section 2. **D1-D4:** visible attachment labels only; original document bodies unavailable.

E1: evidence/host_results.json, actual CPU verification results. **E2:** include/jsfd_core.hpp, tests/host_tests.cpp, src/cuda_demo.cu, CMakeLists.txt, and README.md, newly prepared source. **E3:** evidence/environment.txt, compiler and execution-environment record. All E-items are in the companion ZIP.

External primary references

[R1] NVIDIA. CUDA Programming Guide. Current guide, updated 10 September 2026. [Open source](#).

Primary documentation for CUDA as a general GPU-computing platform.

[R2] NVIDIA. CUDA C++ Programming Guide 12.8.1, texture memory and tex1Dfetch sections. [Open source](#).

Version-pinned API reference for integer-index texture reads and element-type return values.

[R3] NVIDIA. CUDA Programming Guide, §4.14: L2 Cache Control. [Open source](#).

Primary documentation for persisting-access policies, limits, and cache contention.

[R4] NVIDIA. GeForce RTX 50 Series Laptop GPUs — specifications. [Open source](#).

Current product table consulted 18 September 2026; lists RTX 5070 Ti Laptop with 12 GB GDDR7.

[R5] cuRobo. Using in Python — Collision Checking. [Open source](#).

Primary documentation for robot/world collision queries; note its documented sign convention.

[R6] Braithwaite, W., and Museth, K. Accelerating OpenVDB on GPUs with NanoVDB. NVIDIA Technical Blog, 20 August 2020. [Open source](#).

Author/developer description of GPU-oriented volumetric data structures.

[R7] Müller, T., Evans, A., Schied, C., and Keller, A. Instant Neural Graphics Primitives with a Multiresolution Hash Encoding. ACM TOG / SIGGRAPH, 2022. [Open source](#).

Authors' project page; DOI: 10.1145/3528223.3530127.

[R8] Petersen, F., Kuehne, H., Borgelt, C., Welzel, J., and Ermon, S. Convolutional Differentiable Logic Gate Networks. NeurIPS 2024. [Open source](#).

Primary proceedings abstract describing Boolean-gate inference.

[R9] Mordvintsev, A., Randazzo, E., Niklasson, E., and Levin, M. Growing Neural Cellular Automata. Distill, 2020. [Open source](#).

Primary interactive research article on learned growth and damage-recovery dynamics.

[R10] Tero, A., et al. Rules for Biologically Inspired Adaptive Network Design. Science 327, 439–442, 2010. [Open source](#).

Authors' institutional record and abstract; DOI: 10.1126/science.1177894.

[R11] Song, K., et al. Journey of water in pine cones. Scientific Reports 5, 9963, 2015. [Open source](#).

Primary article on water transport and pine-cone mechanics; DOI: 10.1038/srep09963.

[R12] Autumn, K., et al. Evidence for van der Waals adhesion in gecko setae. PNAS 99, 12252–12256, 2002. [Open source](#).

Primary article bibliographic record; DOI: 10.1073/pnas.192252799.

[R13] Qian, L., and Winfree, E. Scaling Up Digital Circuit Computation with DNA Strand Displacement Cascades. Science 332, 1196–1201, 2011. [Open source](#).

Authors' institutional record and abstract; DOI: 10.1126/science.1200520.

[R14] Stérin, T., Eshra, A., Evans, C. G., Adio, J., and Woods, D. A thermodynamically favoured molecular computer. Nature, published online 16 September 2026. [Open source](#).

Primary article text; issue dated 17 September 2026. DOI: 10.1038/s41586-026-10996-5. Text-based review, not figure measurements.

[R15] OpenMM. User Guide, Standard Forces. Current documentation consulted 18 September 2026. [Open source](#).

Primary documentation illustrating force, interaction, and parameter structure in molecular modeling.

[R16] University of Waterloo. Concorde TSP Solver — project home. [Open source](#).

Primary project reference for an exact symmetric traveling-salesman baseline.